

Relative path: convert_repo_to_pdf.py

Filename: convert_repo_to_pdf.py Extension: .py

```
#!/usr/bin/env python3
"""Convert every text file in a repository into searchable PDFs."""

from __future__ import annotations

import argparse
import os
import sys
import tempfile
from collections import defaultdict
from pathlib import Path

from reportlab.lib.pagesizes import A4
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfgen import canvas

PAGE_WIDTH, PAGE_HEIGHT = A4
MARGIN = 42
FONT_NAME = "Courier"
FONT_SIZE = 8
HEADER_SIZE = 10
LEADING = 10
CONTROL_BYTES = set(range(0, 9)) | set(range(14, 32))

def is_within(path: Path, parent: Path) -> bool:
    try:
        path.resolve().relative_to(parent.resolve())
        return True
    except ValueError:
        return False

def is_binary(data: bytes) -> bool:
    """Heuristically classify data without relying on filename extensions."""
    if not data:
        return False

    # UTF-16/UTF-32 text commonly contains NUL bytes but is still text.
    if data.startswith((b"\xff\xfe", b"\xfe\xff", b"\xff\xfe\x00\x00", b"\x00\x00\xfe\xff")):
        return False

    if b"\x00" in data:
        return True

    suspicious = sum(byte in CONTROL_BYTES for byte in data)
    return suspicious / len(data) > 0.02

def read_text(path: Path) -> str:
    """Read as UTF-8, replacing malformed characters as requested."""
    raw = path.read_bytes()
    if is_binary(raw):
        raise ValueError("BINARY_FILE")

    # UTF-8 is the standard path; BOM variants are handled gracefully.
    if raw.startswith(b"\xef\xbb\xbf"):
        return raw.decode("utf-8-sig", errors="replace")
    if raw.startswith((b"\xff\xfe", b"\xfe\xff")):
        return raw.decode("utf-16", errors="replace")

    return raw.decode("utf-8", errors="replace")

def pdf_relative_path(relative_source: Path) -> Path:
    """main.py -> main.pdf; Makefile -> Makefile.pdf."""
    return relative_source.with_suffix(".pdf") if relative_source.suffix else (
        relative_source.parent / f"{relative_source.name}.pdf"
    )

def build_targets(files: list[Path], source: Path, output: Path) -> dict[Path, Path]:
```

```

"""Create unique output names while preserving the source hierarchy."""
grouped: dict[str, list[Path]] = defaultdict(list)

for file_path in files:
    rel = file_path.relative_to(source)
    candidate = pdf_relative_path(rel)
    grouped[candidate.as_posix().casefold()].append(file_path)

targets = {}
for group in grouped.values():
    if len(group) == 1:
        source_file = group[0]
        targets[source_file] = output / pdf_relative_path(source_file.relative_to(source))
    else:
        # Handles e.g. app.py + app.md, which would both otherwise be app.pdf.
        for source_file in group:
            rel = source_file.relative_to(source)
            targets[source_file] = output / rel.parent / f"{rel.name}.pdf"

return targets

def wrap_line(line: str, max_chars: int) -> list[str]:
    """Soft-wrap long physical lines without altering whitespace."""
    if not line:
        return [""]

    return [line[index:index + max_chars] for index in range(0, len(line), max_chars)]

def write_pdf(source_file: Path, relative_path: Path, target: Path) -> None:
    text = read_text(source_file).expandtabs(4)
    target.parent.mkdir(parents=True, exist_ok=True)

    fd, temp_name = tempfile.mkstemp(prefix=".pdf-convert-", suffix=".pdf", dir=target.parent)
    os.close(fd)
    temp_path = Path(temp_name)

    try:
        pdf = canvas.Canvas(str(temp_path), pagesize=A4, pageCompression=1)
        pdf.setTitle(relative_path.name)

        char_width = pdfmetrics.stringWidth("M", FONT_NAME, FONT_SIZE)
        max_chars = max(1, int((PAGE_WIDTH - 2 * MARGIN) / char_width))
        y = PAGE_HEIGHT - MARGIN

        def new_page() -> None:
            nonlocal y
            pdf.showPage()
            pdf.setFont(FONT_NAME, FONT_SIZE)
            y = PAGE_HEIGHT - MARGIN

        # Required metadata at the top of page one.
        pdf.setFont(FONT_NAME, HEADER_SIZE)
        extension = relative_path.suffix or "(no extension)"
        pdf.drawString(MARGIN, y, f"Relative path: {relative_path.as_posix()}")
        y -= 14
        pdf.drawString(MARGIN, y, f"Filename: {relative_path.name}      Extension: {extension}")
        y -= 20

        pdf.setFont(FONT_NAME, FONT_SIZE)
        for physical_line in text.splitlines():
            for rendered_line in wrap_line(physical_line, max_chars):
                if y < MARGIN:
                    new_page()
                pdf.drawString(MARGIN, y, rendered_line)
                y -= LEADING

        # Preserve a final blank line when the source ends with one.
        if text.endswith(("\\n", "\\r")) and y >= MARGIN:
            pdf.drawString(MARGIN, y, "")

        pdf.save()
        os.replace(temp_path, target)
    except Exception:
        temp_path.unlink(missing_ok=True)

```

```

raise

def main() -> int:
    parser = argparse.ArgumentParser(
        description="Recursively convert all text files in a directory to PDFs."
    )
    parser.add_argument("source", type=Path, help="Repository/directory to scan")
    parser.add_argument(
        "output",
        type=Path,
        nargs="?",
        default=Path("pdf_output"),
        help="Directory for generated PDFs (default: pdf_output)",
    )
    args = parser.parse_args()

    source = args.source.resolve()
    output = args.output.resolve()

    if not source.is_dir():
        print(f"Error: source directory does not exist: {source}", file=sys.stderr)
        return 2
    if source == output:
        print("Error: output directory must differ from source directory.", file=sys.stderr)
        return 2

    errors: list[str] = []
    files: list[Path] = []

    def walk_error(error: OSError) -> None:
        errors.append(f"Scan error: {error}")

    for root, directories, filenames in os.walk(source, topdown=True, onerror=walk_error):
        root_path = Path(root)

        # Avoid recursively converting PDFs if output is placed inside source.
        directories[:] = [
            name for name in directories
            if not is_within(root_path / name, output)
        ]

        for filename in filenames:
            path = root_path / filename
            if not is_within(path, output):
                files.append(path)

    files.sort(key=lambda p: p.relative_to(source).as_posix().casefold())
    targets = build_targets(files, source, output)

    converted = 0
    skipped_binary = 0

    for index, source_file in enumerate(files, start=1):
        relative = source_file.relative_to(source)
        print(f"[{index}/{len(files)}] {relative.as_posix()}", flush=True)

        try:
            write_pdf(source_file, relative, targets[source_file])
            converted += 1
        except ValueError as error:
            if str(error) == "BINARY_FILE":
                skipped_binary += 1
                print(" -> skipped (binary)", flush=True)
            else:
                errors.append(f"{relative}: {error}")
                print(f" -> error: {error}", flush=True)
        except Exception as error:
            errors.append(f"{relative}: {type(error).__name__}: {error}")
            print(f" -> error: {type(error).__name__}: {error}", flush=True)

    print("\nSummary")
    print(f" Total files scanned: {len(files)}")
    print(f" Text files converted: {converted}")
    print(f" Binary files skipped: {skipped_binary}")
    print(f" Errors encountered: {len(errors)}")

```

```
if errors:
    print("\nErrors:")
    for error in errors:
        print(f"  - {error}")

return 1 if errors else 0

if __name__ == "__main__":
    raise SystemExit(main())
```